

WeightPCA Samajhne Ke Liye

Matrix Basics se Grokking Predictor Tak — Zero Gap Walkthrough

Jonathan ke liye

October 1, 2026

Contents

1	Roadmap — Hum Kahan Ja Rahe Hain	2
2	Vector Ek Point Nahi, Ek Arrow Hai	2
3	Matrix Ek Grid Nahi, Ek Transformation Hai	2
4	Dot Product Aur Orthogonality	3
5	Matrix Ki Special Directions: Eigenvectors	3
6	Problem: Eigenvectors Har Matrix Ke Liye Kaam Nahi Karte	4
7	Singular Value Decomposition (SVD)	4
8	Rank: Ek Purana Tarika Jo Kaafi Nahi Hai	5
9	Entropy: “Kitna Phela Hua Hai” Naapne Ka Tarika	5
10	Effective Rank: Singular Values Ko Entropy Banana	5
11	NormEffRank: Alag-Alag Size Ke Layers Ko Compare Karna	6
12	Ek Matrix Se Poore Model Tak	7
13	Ek Checkpoint Se Training History Tak: “Signal”	7
14	Signal Se Ek Event Epoch Nikalna	7
15	Hypothesis: Rank Collapse Grokking Ko Predict Kyun Kare	8
16	Tumhare Project Me Kya Actually Hua	8
17	Bada Picture: Iska Matlab Thesis Ke Liye Kya Hai	8

1 Roadmap — Hum Kahan Ja Rahe Hain

Tumhara sawaal tha: WeightPCA kaise kaam karta hai. Is document ka plan simple hai — ek seedhi line me challenge, koi jump nahi. Pehle vector aur matrix ka matlab ek transformation ke roop me dekhenge, fir ek matrix ki “special directions” (eigenvectors, phir singular vectors) samjhenge, fir un directions se ek number nikalna seekhenge jo batata hai matrix kitni directions “use” kar raha hai (effective rank), aur aakhir me ye number tumhare thesis ke WeightPCA predictor me kaise use hota hai aur usne kya result diya.

Har section pichle section par khada hai. Koi bhi naya term tabhi aayega jab previous section use define kar chuka ho.

2 Vector Ek Point Nahi, Ek Arrow Hai

Vector ek list of numbers hai, ye tum jaante ho. Par is document me vector ko sirf list ki tarah nahi, balki ek *arrow* ki tarah dekhna hai — origin se shuru hoke ek point tak jaata hua.

Do vectors ko add karna matlab do arrows ko ek ke baad ek rakhna, resultant arrow naya vector hai. Ek vector ko kisi scalar c se multiply karna matlab usi direction me arrow ko lamba ya chhota karna (agar $c < 0$ to ulta ghuma dena).

Ye do operations — addition aur scalar multiplication — milke ek **vector space** banate hain: saare possible vectors ka collection, jisme ye dono operations defined hain. $\mathbb{R}^2$ (plane) aur $\mathbb{R}^3$ (3D space) sabse common examples hain.

Linear combination: $c_1 v_1 + c_2 v_2 + \dots$ jaisi koi bhi expression, jahan v_i vectors hain aur c_i scalars. Agar kuch vectors $v_1, \dots, v_n$ ke linear combinations se poora vector space ban sakta hai, to unhe us space ka **basis** kehte hain. $\mathbb{R}^2$ ka sabse simple basis hai $e_1 = (1, 0)$ aur $e_2 = (0, 1)$ — kyunki koi bhi $(x, y) = x e_1 + y e_2$ likha ja sakta hai.

Yahan ek zaroori point hai: basis unique nahi hota. Wahi $\mathbb{R}^2$ ka ek doosra basis bhi ho sakta hai, jaise $(1, 1)$ aur $(1, -1)$ — jab tak ye do vectors ek-doesre ke multiple na hon (yaani independent hon), ye bhi poore plane ko cover kar sakte hain. Ye point aage SVD samajhne me kaam aayega: alag-alag kaam ke liye alag basis choose karna faaydemand hota hai.

3 Matrix Ek Grid Nahi, Ek Transformation Hai

Ab tak matrix tumne numbers ke grid ki tarah dekha hoga, jisme row-column operations hote hain. Is document me matrix ko ek alag tarah dekhna hai: ek matrix W ek **function** hai jo ek vector lekar doosra vector deta hai — $x \mapsto Wx$.

Isko concrete banane ke liye ek running example lete hain, jo is poore document me use hoga:

$$W = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}.$$

Koi bhi generic vector lo, jaise $x = (1, 0)$. Isko W se multiply karo:

$$Wx = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

Dekho kya hua: input vector sirf $e_1 = (1, 0)$ direction me tha, par output $(3, 1)$ me dono components hain. Matlab W ne input ko ghuma bhi diya aur stretch bhi kiya — output input ki simple scaled copy nahi hai.

Ye generic behaviour hai: ek random input vector par matrix lagane se output usually ek “mixed” direction me chala jaata hai, input wali direction me nahi rehta. Agle section me dekhenge ki kuch *special* input directions aisi hoti hain jinke saath ye mixing nahi hoti.

4 Dot Product Aur Orthogonality

Do directions ke beech “angle” ko numerically capture karne ke liye dot product chahiye: $u \cdot v = u_1v_1 + u_2v_2 + \dots$. Agar $u \cdot v = 0$, to u aur v **orthogonal** (perpendicular) hain.

Ek vector v ki **length** (ya norm) hoti hai $\|v\| = \sqrt{v \cdot v}$. Agar $\|v\| = 1$, to v **unit vector** hai.

Ek basis jiske saare vectors ek-dosre se orthogonal hon *aur* sab unit vectors hon, use **orthonormal basis** kehte hain. Ye basis special hai kyunki ismein coordinates nikalna sirf dot product lagane jitna simple ho jaata hai, aur lengths/angles distort nahi hote jab tum is basis me vector ko describe karte ho. SVD ka poora structure orthonormal basis par hi khada hai — isliye ye definition yahan zaroori thi.

5 Matrix Ki Special Directions: Eigenvectors

Section 3 me dekha ki generic vector mix ho jaata hai. Ab sawaal ye hai: kya koi aisi input direction hai jo W lagane ke baad apni hi direction me rahe, bas lambi-chhoti ho jaaye?

Agar koi non-zero vector v aur scalar λ aise mil jaayein ki

$$Wv = \lambda v,$$

to v ko W ka **eigenvector** kehte hain, aur λ ko uska **eigenvalue**. Matlab: is direction me W sirf scale karta hai (factor λ se), ghumata nahi.

Apne running example $W = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ par try karte hain. Vector $v_1 = \frac{1}{\sqrt{2}}(1, 1)$ lo:

$$Wv_1 = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 4 \\ 4 \end{pmatrix} = 4 \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = 4v_1.$$

Output bilkul v_1 ki hi direction me hai, sirf 4 guna bada — koi mixing nahi, jo Section 3 ke generic case se bilkul ulta hai. Isi tarah $v_2 = \frac{1}{\sqrt{2}}(1, -1)$ ke liye $Wv_2 = 2v_2$ nikalta hai (khud karke verify kar sakte ho). Ye 4 aur 2 is W ke eigenvalues hain — in numbers ko yaad rakhne ki zaroorat nahi, sirf ye samajhna hai ki har matrix ki apni special directions hoti hain jahan ye seedha scale karta hai.

Humara W symmetric hai (matlab $W = W^T$, upar-neeche mirror). Ek useful fact (spectral theorem, bina proof ke le rahe hain): har symmetric matrix ke eigenvectors *orthogonal* chunne ja sakte hain. Yahi fact agle section me SVD tak pahunchaayega.

6 Problem: Eigenvectors Har Matrix Ke Liye Kaam Nahi Karte

Eigenvector ki definition maangti hai ki W square matrix ho (rows = columns), kyunki Wv aur v same size ke hone chahiye taaki $Wv = \lambda v$ likha ja sake. Tumhare thesis ke weight matrices me se kai non-square hain — jaise `mlp_in` jo 512×128 hai: input 128-dimensional, output 512-dimensional. Yahan “ $Wv = \lambda v$ ” likhna hi possible nahi, kyunki Wv 512-dim hai aur v 128-dim.

Aur jo matrices square bhi hain (jaise `query`, 128×128), wo symmetric nahi hain zaroori nahi ki — aur non-symmetric matrices ke eigenvectors orthogonal nahi hote, kabhi-kabhi eigenvalues complex number bhi aa jaate hain. Isliye ek aisa tool chahiye jo:

- kisi bhi shape ke matrix (square ho ya na ho) par kaam kare,
- hamesha real, orthogonal directions de.

Yahi tool Singular Value Decomposition (SVD) hai.

7 Singular Value Decomposition (SVD)

SVD ka claim ye hai: *koi bhi* matrix W (size $m \times n$) ko teen simple matrices me toda ja sakta hai:

$$W = U\Sigma V^T,$$

jahan:

- V ke columns $v_1, v_2, \dots$ **input space** ($\mathbb{R}^n$) ka ek orthonormal basis hain — inhe **right singular vectors** kehte hain,
- U ke columns $u_1, u_2, \dots$ **output space** ($\mathbb{R}^m$) ka ek orthonormal basis hain — inhe **left singular vectors** kehte hain,
- Σ ek diagonal matrix hai, jiski diagonal par non-negative numbers $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ hain — inhe **singular values** kehte hain.

Is decomposition ka matlab geometrically ye hai: W lagana equivalent hai teen steps karne ke — pehle input ko V ke basis me ghumao, fir har axis ko uske σ_i se scale karo, fir output ko U ke basis me ghumao wapas. Rotation lengths/angles ko distort nahi karta (Section 4 ka orthonormal basis wala point yahi use ho raha hai) — saara “stretching” sirf Σ ke singular values me ho raha hai.

Is decomposition ka sabse important consequence: har right singular vector v_i ke liye,

$$Wv_i = \sigma_i u_i.$$

Yaani Section 5 jaisa hi behaviour — input v_i direction me do, output seedha u_i direction me aata hai, koi mixing nahi, sirf σ_i factor se scale. Bas fark itna hai: ab input aur output alag-alag spaces me ho sakte hain ($u_i \neq v_i$ ho sakta hai), aur ye kisi bhi shape ke matrix ke liye kaam karta hai.

Apne running example par wapas aate hain. $W = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ symmetric hai, isliye iska SVD seedha Section 5 ke eigenvectors se aata hai:

$$v_1 = u_1 = \frac{1}{\sqrt{2}}(1, 1), \quad \sigma_1 = 4, \quad v_2 = u_2 = \frac{1}{\sqrt{2}}(1, -1), \quad \sigma_2 = 2.$$

(Symmetric positive-definite matrix ke liye singular values aur eigenvalues same hote hain, aur $U = V$ ho jaata hai — ye ek special coincidence hai, general matrix me $U \neq V$ hota hai.)

Isiliye jab maine pehle kaha tha “matrix kitni directions use karta hai” — uska matlab tha: kitne σ_i bade (non-negligible) hain. Agar sirf σ_1 bada hai aur baaki sab chhote, to W ka action practically ek hi direction ($v_1 \rightarrow u_1$) tak simit hai. Agar saare σ_i comparable hain, to W apna “effort” kai directions me spread kar raha hai.

8 Rank: Ek Purana Tarika Jo Kaafi Nahi Hai

Matrix ka **rank** hota hai: kitne σ_i strictly non-zero hain (equivalently, output space me kitni independent directions actually use hoti hain). Humare running example me $\text{rank} = 2$ (dono $\sigma_1 = 4, \sigma_2 = 2$ non-zero hain).

Rank ek **hard count** hai — ya to ek σ_i zero hai ya nahi, beech ka kuch nahi. Problem ye hai: real neural network training me singular values kabhi exact zero nahi hote, bas chhote-chhote hote jaate hain. Rank is situation me hamesha full count dega (jaise 2, 2, 2...), chahe ek σ_i dusre se $1000\times$ chhota kyun na ho gaya ho. Matlab rank us gradual “collapse” ko capture nahi kar paata jo training ke dauraan dheere-dheere hota hai.

Isiliye ek aisi quantity chahiye jo rank jaisi hi ho (“kitni directions actually use ho rahi hain”), par *smooth* ho — chhote σ_i ko gradually kam weight de, poora zero hone ka wait na kare. Isko banane ke liye ek detour lena padega: entropy.

9 Entropy: “Kitna Phela Hua Hai” Naapne Ka Tarika

Maan lo tumhare paas R non-negative numbers $p_1, \dots, p_R$ hain jo sum hoke 1 banate hain (ek probability distribution ki tarah). **Entropy** is distribution ki “spread-outness” naapti hai:

$$H(p) = - \sum_{i=1}^R p_i \log p_i.$$

(Convention: $0 \log 0 := 0$, taaki zero-probability wale terms problem na karein.)

Do extreme cases dekho, taaki formula ka matlab clear ho:

Generic/messy case: agar saare p_i barabar hain (yaani $p_i = 1/R$ har i ke liye) — distribution poori tarah spread hua hai. Is case me $H(p) = - \sum_i \frac{1}{R} \log \frac{1}{R} = \log R$, jo is formula ka **maximum possible value** hai.

Special/clean case: agar ek hi $p_j = 1$ hai aur baaki sab 0 — poora mass ek hi jagah concentrated hai. Is case me $H(p) = -1 \cdot \log 1 = 0$, jo **minimum possible value** hai.

Yani: entropy jitni zyada, distribution utna spread hua; entropy jitni kam, distribution utna concentrated. Yahi idea ab singular values par apply karna hai.

10 Effective Rank: Singular Values Ko Entropy Banana

Singular values $\sigma_1, \dots, \sigma_R$ khud probability distribution nahi hain (ye sum hoke 1 nahi banate), par inhe normalize karke banaya ja sakta hai:

$$p_i = \frac{\sigma_i}{\sum_j \sigma_j}.$$

Ab p_i ’s sum hoke 1 banate hain — ye batate hain ki matrix ka total “energy” kitna i -th direction me gaya.

Isi p_i par Section 9 wali entropy formula lagao:

$$\text{EffRank}(W) = - \sum_{i=1}^R p_i \log p_i.$$

Yahi tumhare `weight_pca.py` me likha formula hai (Yunis et al. 2024 ka Equation 1) — bilkul Section 9 ki entropy, bas p_i singular values se banaya gaya hai.

Running example par calculate karte hain: $\sigma_1 = 4, \sigma_2 = 2$, to $\sum \sigma_j = 6$, isliye $p_1 = 4/6 \approx 0.667$ aur $p_2 = 2/6 \approx 0.333$.

$$\text{EffRank}(W) = -(0.667 \log 0.667 + 0.333 \log 0.333) \approx 0.636.$$

Compare karne ke liye, agar ye matrix pura balanced hota ($\sigma_1 = \sigma_2 = 3$, yani $p_1 = p_2 = 0.5$), to $\text{EffRank} = \log 2 \approx 0.693$ hota — maximum possible $R = 2$ ke liye (Section 9 ka generic case).

Ab *collapsed* case dekhte hain, jisse “rank collapse” ka matlab clear ho: maan lo training ke baad $\sigma_1 = 5.9$ aur $\sigma_2 = 0.1$ ho gaye (sum wahi 6, par ab energy ek hi direction me concentrated hai). Tab $p_1 \approx 0.983$, $p_2 \approx 0.017$, aur

$$\text{EffRank}(W) \approx 0.083.$$

Dekho fark: balanced case me $\text{EffRank} \approx 0.693$ tha, collapsed case me ≈ 0.083 — bina kisi σ_i ko exactly zero kiye, EffRank ne ye gradual shift smoothly capture kar liya. Yahi Section 8 ki problem (hard rank ka “all or nothing” hona) yahan solve ho gayi.

11 NormEffRank: Alag-Alag Size Ke Layers Ko Compare Karna

Ek subtlety hai: EffRank ki maximum possible value R par depend karti hai ($\log R$, Section 9 se). Tumhare thesis ke layers alag-alag size ke hain — `query` ka $R = 128$, `mlp_in` ka $R = 128$ bhi (chhota dimension hi rank decide karta hai), `output_head` ka $R = 114$. Agar tum seedha EffRank compare karoge do alag R wale layers ke beech, to comparison fair nahi hoga — bada R wala layer hamesha bada EffRank allow karta hai, chahe wo utna hi “collapsed” kyun na ho jitna chhota layer.

Isiliye normalize karte hain:

$$\text{NormEffRank}(W) = \frac{\text{EffRank}(W)}{R}.$$

Ek zaroori correction jo tumhare code ke comments khud flag karte hain: NormEffRank ki range $[0, 1]$ nahi hai jaisa naam se lag sakta hai. Maximum entropy $\log R$ hoti hai (poora R nahi), isliye NormEffRank ki actual range hai $\left[0, \frac{\log R}{R}\right]$. Tumhare layers ke liye ($R = 114$ se 512 tak), ye upper bound roughly 0.01 se 0.05 ke beech aata hai. Is number ko yaad rakhna zaroori nahi, bas itna samajhna hai ki NormEffRank ek *chhoti* si value hogi, 1 ke paas kabhi nahi jaayegi — warna thresholds padhte waqt confuse ho sakte ho.

12 Ek Matrix Se Poore Model Tak

Ab tak hum ek single weight matrix W ki baat kar rahe the. Tumhare **TransformerFourHead** model me kai weight matrices hain, par WeightPCA sabko nahi dekhta — sirf wahi 2-D Linear layers jo HTSR Alpha predictor bhi dekhta hai (consistency ke liye, taaki predictors aapas me compare ho sakein):

Layer	Shape (R = chhota dimension)
query	128×128
key	128×128
value	128×128
mlp_in	512×128
mlp_out	128×512
output_head	114×128

Embedding tables (token/position) **nn.Embedding** hain, Linear layer nahi — isliye inhe skip kiya gaya hai, HTSR ke precedent ko follow karte hue. Biases vectors hain, matrices nahi — SVD unpar define hi nahi hota, isliye unhe bhi skip kiya jaata hai.

Har checkpoint par, in 6 matrices me se har ek ka apna SVD, apna EffRank, apna NormEffRank nikalta hai. Fir in 6 NormEffRank values ka **mean** le liya jaata hai — yahi ek checkpoint ke liye poore model ka *single* summary number banta hai.

13 Ek Checkpoint Se Training History Tak: “Signal”

Training ke dauraan model ke weights baar-baar checkpoint hote hain (tumhare project me 401–801 checkpoints per seed). Har checkpoint par Section 12 wala mean NormEffRank nikal lo. Ye sab checkpoints ke across ek time series ban jaati hai:

$$\text{signal} = (s(t_1), s(t_2), \dots, s(t_K)),$$

jahan $t_1 < t_2 < \dots < t_K$ checkpoint epochs hain aur $s(t_k)$ us epoch par mean NormEffRank hai. Yahi “signal” hai jo WeightPCA predictor training ke poore course me track karta hai — bilkul usi tarah jaise test accuracy ka curve track hota hai, bas ye curve weight-structure ka hai, performance ka nahi.

14 Signal Se Ek Event Epoch Nikalna

Predictor ka kaam sirf signal banana nahi hai — usse ek specific **epoch** nikalna hai jise “grokking ka signal” bola ja sake. Tumhare code me teen tarike hain:

Primary rule (jo official verdict decide karta hai): *global minimum epoch* — wo epoch jahan signal $s(t_k)$ apni sabse chhoti value par pahunchta hai, poori training history me. Idea: jab weights sabse zyada “collapsed” hon, wahi event hai.

Exploratory rule 1: *steepest drop epoch* — wo epoch jahan consecutive checkpoints ke beech signal sabse zyada ek-saath gira ho (largest one-step decrease).

Exploratory rule 2: *90% threshold-crossing epoch* — signal apni starting value se apne global minimum tak jo total gira (“total decline”), uska 90% cover karne wala pehla epoch.

Exploratory rules sirf thesis ke appendix/sensitivity-analysis ke liye hain — final PASS/FAIL verdict sirf primary rule (global minimum) se decide hota hai.

15 Hypothesis: Rank Collapse Grokking Ko Predict Kyun Kare

Ye poora predictor ek specific paper se aaya hai — Yunis et al. 2024, “*Approaching Deep Learning through the Spectral Dynamics of Weights*” (arXiv:2408.11804), Section 3. Unka finding tha: ek single-layer Transformer jo modular addition seekh raha tha (bilkul tumhare Nanda-Unified setup jaisa), usme validation loss ka achanak girna *usi waqt* hota hai jab singular values low-rank behaviour dikhana shuru karte hain — matlab weights kuch hi directions me simit ho jaate hain.

Ek zaroori cautious point: paper khud kehta hai ki ye dono events “coincide” karte hain (saath-saath hote hain), ye nahi kehta ki rank collapse *pehle* hota hai. Tumhare thesis ka protocol specifically ek **leading** signal dhoondh raha hai — jo grokking se pehle warning de de. Yani ye bilkul possible tha ki ye predictor coincide to kare, par lead na kare.

16 Tumhare Project Me Kya Actually Hua

Yahi bilkul hua. Tumhare 5 seeds par (grok epochs 12987, 6921, 9510, 11019, 23747), NormEffRank signal har seed me gira — shuru me roughly 0.0373 se ant tak roughly 0.016–0.019 tak. Par signal ka global minimum *grok epoch ke baad* aaya, har ek 5 seeds me — $1.52\times$ se $4.03\times$ grok epoch tak late.

Frozen protocol ke teen thresholds the:

- $\text{lead_frac} \geq 0.8$ chahiye tha — mila 0.0000.
- $\rho \geq 0.9$ chahiye tha — mila 0.80.
- $\text{mean_rel_lead} \in [0.05, 0.9]$ chahiye tha — mila -1.9029 (negative matlab lead nahi, lag hai).

Teeno thresholds fail hue. **Verdict: FAIL, CLOSED, negative.** Weight-PCA tumhare benchmark ka chhatha (6th) predictor hai jo is stricter, leading-signal-wale protocol me fail hua hai (L2 Norm, Dropout, Spectral, AGE, aur HTSR Alpha ke baad).

17 Bada Picture: Iska Matlab Thesis Ke Liye Kya Hai

Ye result “failure” nahi hai — ye khud ek finding hai. Yunis et al. ka paper sahi tha apni jagah par (rank collapse aur grokking *coincide* karte hain), par tumhara stricter test — jo *leading* indicator maangta hai, sirf coincidence nahi — wo is predictor ko (aur ab tak 5 doosre published predictors ko) pass nahi karne deta.

Agar baaki bache 2 predictors (Higher-MI, Commutator Defect) bhi isi tarah fail hote hain, to tumhare thesis ka actual contribution ye banega: *koi bhi published predictor, is unified aur stricter protocol ke under, genuine ex-ante early-warning signal nahi deta.* Ye apne aap me ek valid, defensible scientific finding hai — pipeline ka bug nahi.